



Mahidol University International College

7-ELEVEN DATABASE FOUNDATIONS PROJECT

Final Project Report

Aryan Gurnani (6780183)

Ayush Tripathi (6780185)

Pinyada Suthisopha-aphorn (6580511)

1. Executive Summary

This report documents the design, implementation, and validation of a relational database supporting the core operations of a multi-branch 7-Eleven convenience store network. The system centralizes product and category data, per-branch inventory with expiration-batch tracking, supplier delivery logistics, point-of-sale transactions, the All Member loyalty program, store operating expenses, and employee shift scheduling.

The database runs on PostgreSQL and is organized into four functional modules: Core Infrastructure, Inventory & Supplier Logistics, Financials & Loyalty, and Workforce Management. Each one is backed by normalized tables, constraint-based validation, trigger-driven automation, role-based access, and its own audit logging.

1.1 Team & Responsibilities

Member	Student ID	Role	Primary Responsibilities
Aryan Gurnani	6780183	Project Documentation & UI Analyst	Captured UI screens, designed staff-facing screen mockups, authored this report and mapped screens to functions
Ayush Tripathi	6780185	Database Architect & Deployment Lead	Designed and normalized the schema, wrote optimized functions, triggers, and views
Pinyada Suthisopha-aphorn	6580511	Repository Administrator & Presentation Lead	Maintained version control and repo structure, organized files, designed the presentation

1.2 Business Domain

7-Eleven is a multi-branch convenience store chain selling food, drinks, electronics, medicinal, hygiene, household, and stationery products. The system focuses on managing localized product catalogs, multi-branch inventory syncing, point-of-sale transactions, store maintenance costs, employee payroll and scheduling, All Member loyalty tracking, and supplier delivery logistics.

1.3 Key Features Implemented

- **Core Infrastructure:** Universal product data, pricing, product categories, and branch location registries.
- **Inventory & Supplier Logistics:** Multi-company delivery tracking (driver and vehicle data), localized branch stock quantities, and expiration-date tracking.
- **Financials & Loyalty Programs:** Point-of-sale transactions across payment methods, real-time stock adjustment at checkout, store expenses, and All Member points tracking.
- **Workforce Management:** Employee roles, attendance/shift status, and shift scheduling.

1.4 Scope Exemptions

- 7-Eleven Delivery (last-mile courier ordering)
- In-app banking
- Front-end UI code and user authentication/session management

2. Database Design

The schema is organized into fourteen core tables plus four audit tables, grouped into the four functional modules described above. All tables use surrogate integer primary keys except for batch-tracking junction tables (store_inventory, delivery_items, transaction_items), which use composite keys of (reference_id, product_id, expiration_date) to correctly model that the same product can exist in multiple expiration batches at the same store.

2.1 Entity-Relationship Diagram

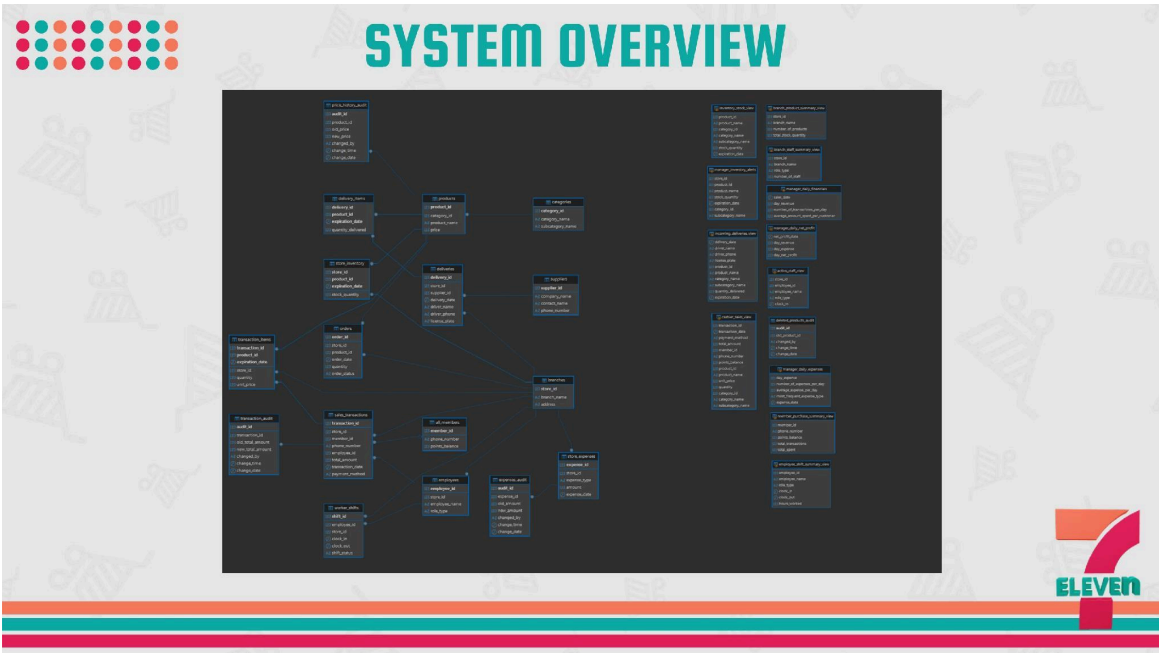


Figure 1. Entity-relationship diagram covering all core, inventory, financial, and workforce tables, plus the audit and reporting-view layer.

2.2 Schema Modules

Module	Tables
Core	branches, categories, products
Inventory & Supplier Logistics	store_inventory, suppliers, orders, deliveries, delivery_items
Financials & Loyalty	all_members, sales_transactions, transaction_items, store_expenses
Workforce Management	employees, worker_shifts
Audit & Security	price_history_audit, deleted_products_audit, expenses_audit, transaction_audit

2.3 Normalization Notes

- All tables satisfy 3NF: every non-key attribute depends only on the whole primary key, and no transitive dependencies exist between non-key columns.
- categories stores category_name and subcategory_name together as a leaf-level entry, constrained UNIQUE per pair. We simplified this from a two-level category/subcategory hierarchy because the catalog never needs to query a parent category on its own.

- store_inventory, delivery_items, and transaction_items use composite primary keys of (store/delivery/transaction id, product_id, expiration_date) rather than a surrogate key, since the natural key already uniquely and meaningfully identifies a batch row, and the composite key doubles as the most common lookup index.

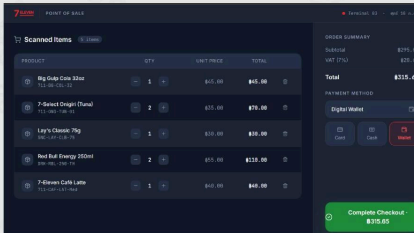
3. Screens, Functions & Example Results

Each subsection below maps a captured or designed screen to the database tables, views, and functions that supply its data, and shows a real example result generated by running the function or view against the seeded database.

3.1 Point of Sale (Cashier Checkout)

Sales Transactions, Inventory & Supplier Deliveries

- Target User: Store Cashier
- Database Tables: sales_transactions & transaction_items (Includes **expiration_date** mapping)
- Data Integrity (Constraints):
 - **chk_valid_payment_method**: UI strictly limited to CASH, CREDIT_CARD, DEBIT_CARD or TRUE_MONEY.
 - **chk_positive_quantity**: Blocks negative or zero quantities (prevents scanning “-1” items).
- Background Automation (Triggers):
 - **trg_update_stock**: Automatically deducts sold items from store inventory.
 - **trg_all_member_points**: Calculates and adds loyalty points based on total purchase amount.
 - **trg_stock_reorder**: Automatically generates a supplier order if item inventory drops below 10.



The screenshot displays a POS system interface. The top section, titled 'Scanned Items', lists five items with their quantities, unit prices, and totals. The bottom section, titled 'Payment Method', shows the total amount due and options for payment, including 'Cash', 'Card', and 'Credit'. A green button at the bottom right indicates 'Complete Checkout'.

PRODUCT	QTY	UNIT PRICE	TOTAL
Big Gulp Code Size Tall 500 mL	1	\$45.00	\$45.00
7-Select Orange (Tall) Tall 500 mL	2	\$35.00	\$70.00
Levi's Classic T-Shirt Tall 500 mL	1	\$80.00	\$80.00
Red Bull Energy 500mL Tall 500 mL	2	\$45.00	\$90.00
7-Select Code Large Tall 500 mL	1	\$40.00	\$40.00

ORDER SUMMARY
SUBTOTAL \$275.00
TAX (7%) \$19.25
Total \$294.25
PAYMENT METHOD
Digital Wallet
Cash Card Credit
Complete Checkout \$294.25

Screen used by: Store Cashier

Database objects: sales_transactions, transaction_items, store_inventory (via trg_update_stock), all_members (via trg_all_member_points)

Function/View: `fn_search_available_products(store_id, keyword)`

```
SELECT * FROM fn_search_available_products(1, 'chip');
```

Example result:

product_id	product_name	stock_quantity	price
2	Lays Rock Salt Chips 50g	8	25.00

Used by the cashier terminal's product search box. The match is a case-insensitive substring search (ILIKE-style), so it also picks up partial words inside longer names. We chose that on purpose: a fast, forgiving search box beats a strict word-boundary match here.

3.2 All Member Digital Wallet



All Member Digital Wallet

- Target User: Customer
- Database View: **member_purchase_summary_view**
- Performance: Instantly loads active points, total visits, and lifetime spend via a pre-calculated view rather than running slow, complex queries.
- Data Integrity (Constraints):
 - chk_positive_points_balance**: Guarantees point balance cannot drop below zero. If an overdraft is attempted, the database blocks it, allowing the UI to safely default to 0 points.




Screen used by: Customer

Database objects: all_members, sales_transactions (via member_purchase_summary_view)

Function/View: member_purchase_summary_view

```
SELECT * FROM member_purchase_summary_view LIMIT 5;
```

Example result:

member_id	phone_number	points_balance	total_transactions	total_spent
1	+66-81-123-4501	150	1	39.00
2	+66-81-123-4502	340	1	50.00
4	+66-81-123-4504	1250	1	40.00

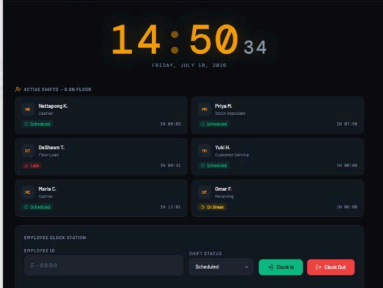
The view pre-joins members with their transaction history so the wallet screen loads instantly with points balance, visit count, and lifetime spend, without the app running a live aggregate query on every open.

3.3 Employee Shift Clock Station



Employee Shifts

- Target User: Store Employees and Branch Manager
- Database Objects: **worker_shifts (Table)** & **active_staff_view (View)**
- Core Function: Back-room tablet instantly displays active floor staff (where **clock_in** is recorded but **clock_out** is NULL).
- Data Integrity (Constraints):
 - Status Validation: **chk_valid_shift_status** strictly limits states to SCHEDULED, ACTIVE, COMPLETED, ABSENT, or CANCELED to prevent invalid custom codes.
 - Time-Theft Protection: **chk_valid_shift_timeline** outright rejects any attempts to submit a "Clock Out" time that occurred before the "Clock In" time.




Screen used by: Store Employees & Branch Manager

Database objects: worker_shifts, employees (via active_staff_view)

Function/View: fn_get_active_staff(store_id)

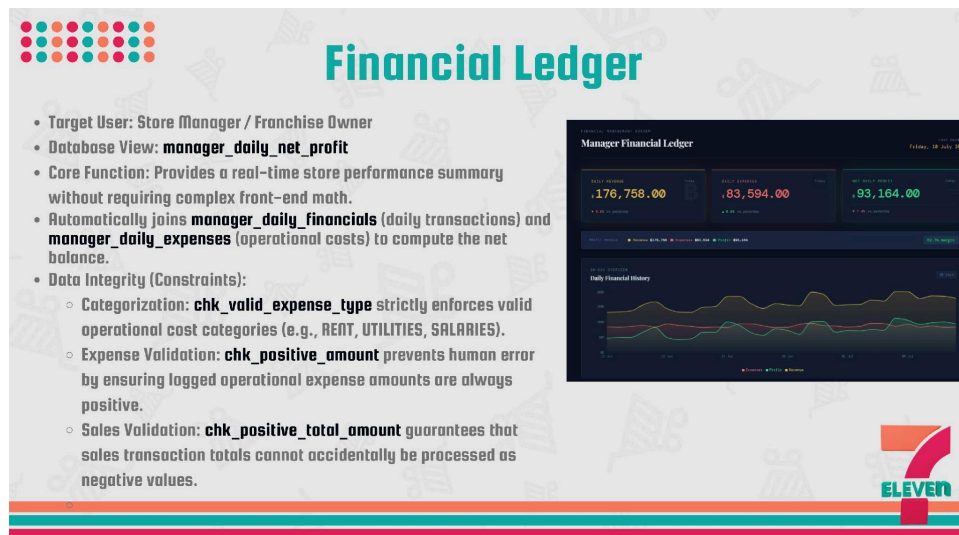
```
SELECT * FROM fn_get_active_staff(1);
```

Example result:

employee_name	role_type	clock_in
Thaksin Security	SECURITY	2026-07-11 00:00:00+00
Piyapat Cashier	CASHIER	2026-07-11 07:00:00+00

Powers the back-room tablet's live "who is currently on the floor" panel by finding shifts with a recorded clock_in and a NULL clock_out. This relies on worker_shifts.clock_out being a nullable column, since a shift still in progress simply has no clock-out time yet.

3.4 Financial Ledger Dashboard



Screen used by: Store Manager / Franchise Owner

Database objects: sales_transactions, store_expenses (via manager_daily_financials, manager_daily_expenses, manager_daily_net_profit)

Function/View: manager_daily_net_profit

```
SELECT * FROM manager_daily_net_profit ORDER BY net_profit_date DESC LIMIT 3;
```

Example result:

net_profit_date	day_revenue	day_expense	day_net_profit
2026-07-11	956.00	366300.00	-365344.00
2026-07-10	4738.00	780700.00	-775962.00
2026-07-09	406.00	4500.00	-4094.00

3.5 Corporate Security & Audit Log

Screen used by: Regional Corporate Auditor / Franchise Owner

Function/View: trg_price_audit -> fn_price_audit()

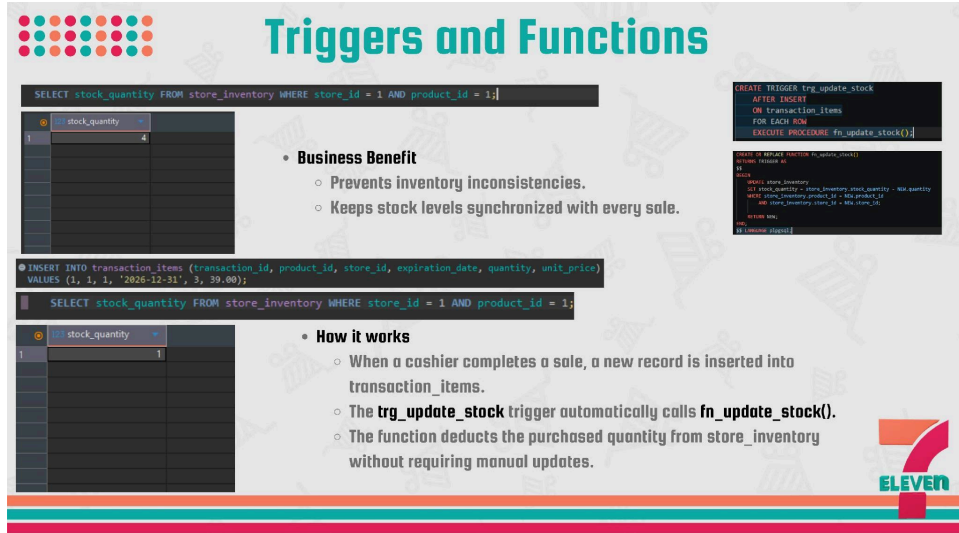
Example result:

The audit layer runs entirely at the database level through AFTER triggers, so it catches changes no matter which client made them, including a direct psql session. That's the right trust boundary for a tamper log: it doesn't rely on any particular application being well-behaved.

4. Trigger-Driven Automation

A chain of four triggers automates the full restock lifecycle without any manual intervention, moving from a single sale to an automatically generated supplier order and back to restored stock.

4.1 Sale → Stock Deduction



Triggers and Functions

```
SELECT stock_quantity FROM store_inventory WHERE store_id = 1 AND product_id = 1;
```

stock_quantity
4

- **Business Benefit**
 - Prevents inventory inconsistencies.
 - Keeps stock levels synchronized with every sale.

```
INSERT INTO transaction_items (transaction_id, product_id, store_id, expiration_date, quantity, unit_price)
VALUES (1, 1, 1, '2026-12-31', 3, 39.00);
```

```
SELECT stock_quantity FROM store_inventory WHERE store_id = 1 AND product_id = 1;
```

stock_quantity
1

- **How it works**
 - When a cashier completes a sale, a new record is inserted into transaction_items.
 - The **trg_update_stock** trigger automatically calls **fn_update_stock()**.
 - The function deducts the purchased quantity from store_inventory without requiring manual updates.

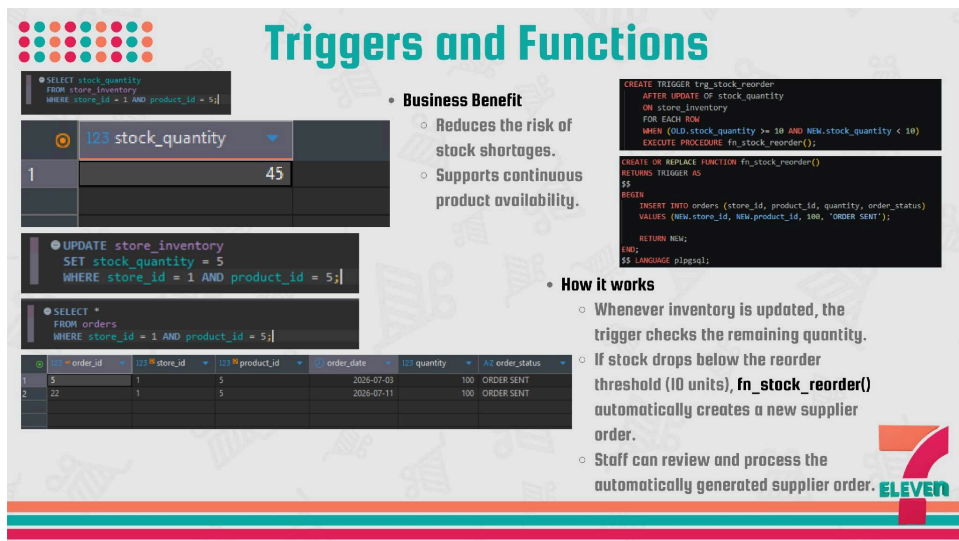
```
CREATE TRIGGER trg_update_stock
AFTER INSERT
ON transaction_items
FOR EACH ROW
EXECUTE PROCEDURE fn_update_stock();
```

```
CREATE OR REPLACE FUNCTION fn_update_stock()
RETURNS VOID AS
$$
BEGIN
    UPDATE store_inventory
    SET stock_quantity = store_inventory.stock_quantity - NEW.quantity
    WHERE store_inventory.product_id = NEW.product_id
    AND store_inventory.store_id = NEW.store_id;
END;
$$ LANGUAGE plpgsql;
```



trg_update_stock (AFTER INSERT ON transaction_items) calls fn_update_stock(), which decrements store_inventory.stock_quantity by the sold quantity for the matching store and product. This keeps on-hand stock synchronized with every sale without the point-of-sale application issuing a separate UPDATE.

4.2 Low-Stock Detection → Automatic Reorder



Triggers and Functions

```
SELECT stock_quantity
FROM store_inventory
WHERE store_id = 1 AND product_id = 5;
```

stock_quantity
45

- **Business Benefit**
 - Reduces the risk of stock shortages.
 - Supports continuous product availability.

```
UPDATE store_inventory
SET stock_quantity = 5
WHERE store_id = 1 AND product_id = 5;
```

```
SELECT *
FROM orders
WHERE store_id = 1 AND product_id = 5;
```

order_id	store_id	product_id	order_date	quantity	order_status
5	1	5	2026-07-03	100	ORDER SENT
22	1	5	2026-07-11	100	ORDER SENT

```
CREATE TRIGGER trg_stock_reorder
AFTER UPDATE OF stock_quantity
ON store_inventory
FOR EACH ROW
WHEN (OLD.stock_quantity >= 10 AND NEW.stock_quantity < 10)
EXECUTE PROCEDURE fn_stock_reorder();
```

```
CREATE OR REPLACE FUNCTION fn_stock_reorder()
RETURNS TRIGGER AS
$$
BEGIN
    INSERT INTO orders (store_id, product_id, quantity, order_status)
    VALUES (NEW.store_id, NEW.product_id, 100, 'ORDER SENT');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

- **How it works**
 - Whenever inventory is updated, the trigger checks the remaining quantity.
 - If stock drops below the reorder threshold (10 units), **fn_stock_reorder()** automatically creates a new supplier order.
 - Staff can review and process the automatically generated supplier order.



trg_stock_reorder (AFTER UPDATE OF stock_quantity ON store_inventory) fires only on the threshold crossing (stock_quantity drops from ≥ 10 to < 10), calling fn_stock_reorder() to insert a new row into orders with status 'ORDER SENT'. Restricting the trigger to the crossing, rather than every update while already below 10, avoids generating a duplicate order on every subsequent sale of an already-low item.

4.3 Order Completion → Stock Restored

Triggers and Functions

```
SELECT stock_quantity FROM store_inventory WHERE store_id = 1 AND product_id = 2;
```

stock_quantity
0

```
INSERT INTO orders (store_id, product_id, quantity, order_status)
VALUES (1, 2, 50, 'IN PROCESS');
```

```
UPDATE orders
SET order_status = 'COMPLETED'
WHERE store_id = 1 AND product_id = 2 AND order_status = 'IN PROCESS';
```

```
SELECT stock_quantity FROM store_inventory WHERE store_id = 1 AND product_id = 2;
```

stock_quantity
50

- **Business Benefit**
 - Eliminates manual inventory updates.
 - Ensures stores always display accurate stock levels.

```
CREATE TRIGGER trg_increase_stock_due_to_order_completion
AFTER UPDATE OF order_status
ON orders
FOR EACH ROW
WHEN (NEW.order_status = 'COMPLETED')
EXECUTE PROCEDURE fn_increase_stock_due_to_order_completion();
```

```
CREATE OR REPLACE FUNCTION fn_increase_stock_due_to_order_completion
RETURNS TRIGGER AS
$$
BEGIN
  UPDATE store_inventory
  SET stock_quantity = store_inventory.stock_quantity + NEW.quantity
  WHERE (store_inventory.product_id = NEW.product_id
  AND store_inventory.store_id = NEW.store_id);
RETURN NEW;
END;
```

- **How it works**
 - When an order status changes to **COMPLETED**, the trigger executes **fn_increase_stock_due_to_order_completion()**.
 - The function increases the inventory quantity for the corresponding store and product.
 - Stock is updated automatically when the order status changes to **COMPLETED**.

trg_increase_stock_due_to_order_completion (AFTER UPDATE OF order_status ON orders, WHEN order_status = 'COMPLETED') calls fn_increase_stock_due_to_order_completion() to add the received quantity back into store_inventory, closing the loop from low-stock alert to restocked shelf.

4.4 Sale → Loyalty Points

Triggers and Functions

```
SELECT member_id, phone_number, points_balance
FROM all_members
WHERE member_id = 1;
```

member_id	phone_number	points_balance
1	+66-81-123-4501	155

```
INSERT INTO sales_transactions (store_id, member_id, phone_number, employee_id, total_amount, payment_method)
VALUES (1, 1, '+66-81-123-4501', 1, 100.00, 'CASH');
```

```
SELECT member_id, phone_number, points_balance
FROM all_members
WHERE member_id = 1;
```

member_id	phone_number	points_balance
1	+66-81-123-4501	159

- **Business Benefit**
 - Removes manual point calculations.
 - Guarantees accurate and consistent loyalty rewards.

```
CREATE TRIGGER trg_all_member_points
AFTER INSERT
ON sales_transactions
FOR EACH ROW
WHEN (NEW.member_id IS NOT NULL)
EXECUTE PROCEDURE fn_all_member_points();
```

```
CREATE OR REPLACE FUNCTION fn_all_member_points
RETURNS TRIGGER AS
$$
BEGIN
  UPDATE all_members
  SET points_balance = (points_balance + FLOOR(NEW.total_amount/25))
  WHERE member_id = NEW.member_id
  AND phone_number = NEW.phone_number;
RETURN NEW;
END;
```

- **How it works**
 - After a sales transaction is inserted, the trigger runs **fn_all_member_points()**.
 - The function calculates reward points from the purchase amount.
 - The member's point balance is updated automatically.

trg_all_member_points (AFTER INSERT ON sales_transactions, WHEN member_id IS NOT NULL) calls fn_all_member_points(), which credits $\text{FLOOR}(\text{total_amount} / 25)$ points to the member's balance. Because of the WHEN guard, the trigger simply doesn't fire on a guest checkout with no member_id, so fn_all_member_points() never has to handle a NULL case itself.

5. Reusable Database Functions

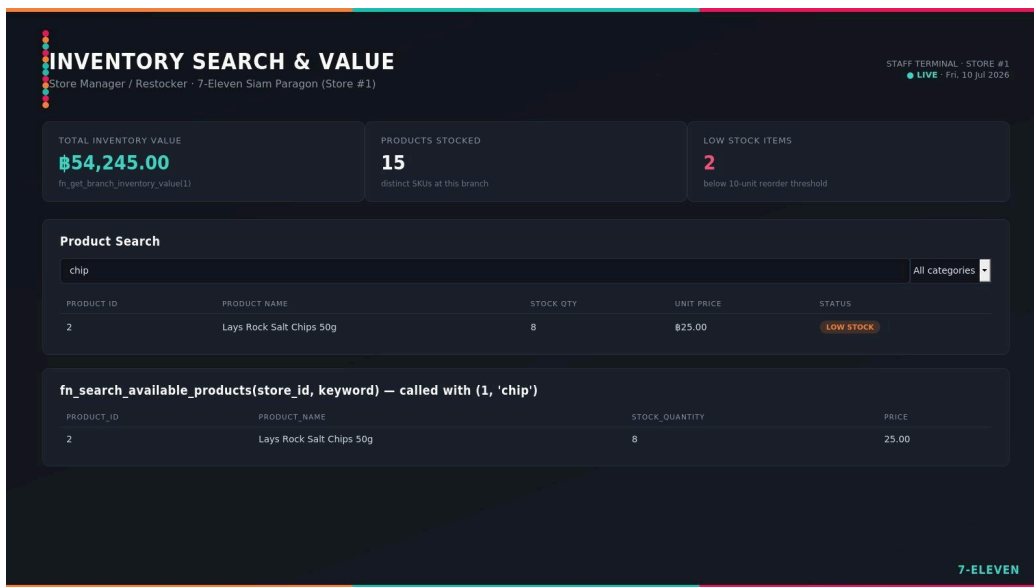
Beyond the trigger-bound functions above, the following stand-alone functions serve repeated query patterns across the reporting and operational screens, avoiding hand-written SQL being duplicated across the application layer.

Business Area	Function	Purpose
Product & Inventory	fn_search_available_products(store_id, keyword)	Searches in-stock products at a branch matching a keyword
Inventory Management	fn_get_branch_inventory_value(store_id)	Calculates the total retail value of a branch's current inventory
Financial Reporting	fn_get_branch_revenue(store_id, start, end)	Sums sales revenue for a branch over a date range
Financial Reporting	fn_get_branch_expenses(store_id, start, end)	Sums operating expenses for a branch over a date range
Financial Reporting	fn_get_branch_profits(store_id, start, end)	Computes revenue minus expenses for a branch over a date range
Inventory Safety	fn_check_expired_products(store_id)	Finds in-stock batches whose expiration_date has passed
Workforce Management	fn_get_employee_schedule(employee_id)	Returns an employee's shift history, most recent first
Workforce Management	fn_get_active_staff(store_id)	Returns staff currently clocked in at a branch
Workforce Management	fn_get_shift_hours(employee_id, start, end)	Totals worked hours for an employee over a date range

5.1 Staff-Facing Screens for These Functions

The reusable functions and reporting views above are not called directly by end customers; they back internal staff tools that do not exist in the public 7-Eleven app. The nine screens below give every remaining function and view in the schema a concrete UI, matching the visual language of the corporate and manager screens shown earlier for the point of sale, member wallet, shift clock, financial ledger, and audit log. They're grouped roughly from single-product lookups through to cross-branch corporate rollups.

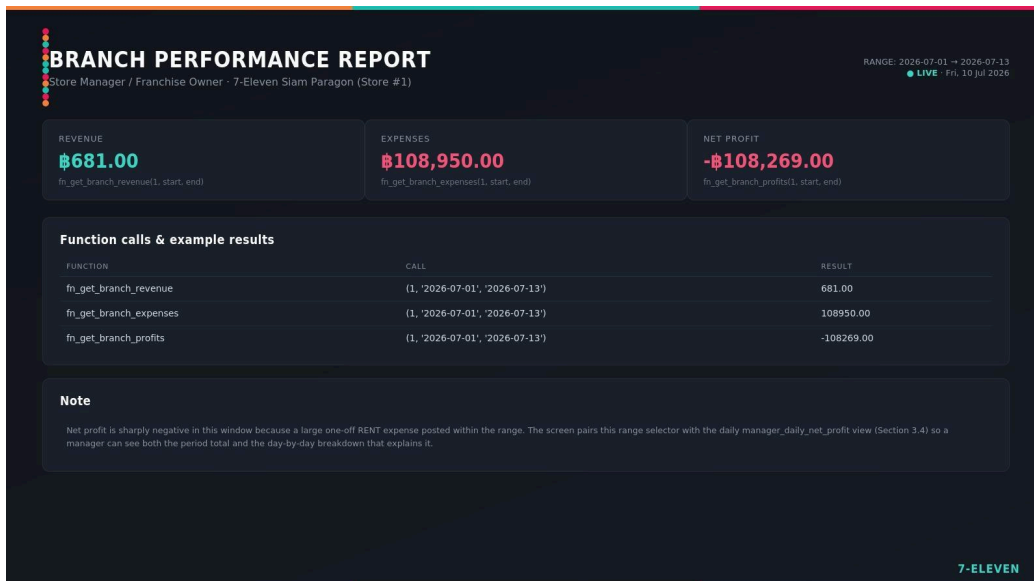
5.1.1 Inventory Search & Value



Functions used: `fn_search_available_products(store_id, keyword)`, `fn_get_branch_inventory_value(store_id)`

Built for a restocker or store manager. A restocker types a keyword into the search box to check on-shelf stock and price before pointing a customer to an aisle. The summary cards at the top give the manager a running total retail value of everything currently on the shelves, so nobody has to sum `store_inventory` by hand.

5.1.2 Branch Performance Report



Functions used: `fn_get_branch_revenue(store_id, start, end)`, `fn_get_branch_expenses(store_id, start, end)`, `fn_get_branch_profits(store_id, start, end)`

For a store manager or franchise owner running a weekly or monthly review. A date-range picker lets them pull revenue, expenses, and net profit for any custom period, sitting alongside the always-on daily dashboard shown with the Financial Ledger screen.

5.1.3 Expired Stock Alert

EXPIRED STOCK ALERT

Store Manager / Restocker - 7-Eleven Siam Paragon (Store #1)

AUTO-REFRESHED DAILY
● LIVE · Fri, 10 Jul 2026

EXPIRED BATCHES

3

fn_check_expired_products(1)

UNITS AFFECTED

80

sum of stock_quantity across expired batches

ACTION REQUIRED

Pull & Log Waste

remove from shelf before next audit

fn_check_expired_products(store_id) — called with (1)

PRODUCT ID	PRODUCT NAME	STOCK QTY	EXPIRATION DATE	STATUS
1	Ezygo Pork Gyoza Box	5	2026-07-11	EXPIRED
3	Nestle Extreme Chocolate Cone	50	2026-07-11	EXPIRED
26	Jade Dragon Minced Pork Bun	25	2026-07-14	EXPIRES TODAY

Raw query result

STORE_ID	PRODUCT_ID	STOCK_QUANTITY	EXPIRATION_DATE
1	1	5	2026-07-11
1	3	50	2026-07-11
1	26	25	2026-07-14

7-ELEVEN

A store manager or restocker uses this to catch what a routine walk-through might miss. It surfaces every batch whose `expiration_date` has already passed, or expires today, so it can be pulled from the shelf and logged as waste before the next stock audit.

RANGE: 2026-07-01 → 2026-07-13

LIVE Fri, 10 Jul 2026

EMPLOYEE SCHEDULE & HOURS

Branch Manager / HR • Employee: Somchai Manager (ID #1)

fn_get_employee_schedule(employee_id) — called with (1)

SHIFT ID	CLOCK IN	CLOCK OUT	STATUS
26	2026-07-11 08:00	2026-07-11 16:00	COMPLETED
34	2026-07-10 08:00	2026-07-10 16:00	COMPLETED
3	2026-07-09 08:00	2026-07-09 16:00	COMPLETED

TOTAL HOURS (PIYAPAT CASHIER, ID #9)

16.0 hrs

fn_get_shift_hours(9, '2026-07-01', '2026-07-13')

Function reference

FUNCTION	PURPOSE
fn_get_employee_schedule	Full shift history, most recent first
fn_get_active_staff	Staff currently clocked in (see Section 3.3)
fn_get_shift_hours	Total worked hours over a date range

Branch managers and HR use this for payroll review: look up a single employee's full shift history and total their worked hours over a chosen date range. The Employee Shift Clock Station answers who is on the floor right now; this answers who worked in the past.

Database objects: orders table (populated by fn_stock_reorder, advanced by fn_increase_stock_due_to_order_completion), incoming_deliveries_view

The low-stock-to-reorder trigger chain automatically creates an order and, once it's marked complete, restocks the shelf, but neither of those triggers has a screen of its own. This is where a store manager actually views auto-generated orders and marks one COMPLETED once goods arrive, which is the action that fires trg_increase_stock_due_to_order_completion. Alongside it sits a live feed of incoming_deliveries_view, showing which supplier, driver, and vehicle is bringing each shipment in.

5.1.6 Low Stock & Expiry Alerts

View used: manager_inventory_alerts

This is a store manager's early-warning screen, distinct from the strictly already-expired list shown in the Expired Stock Alert screen. It flags anything at risk before it becomes a problem, stock at or below 10 units, or expiring within 7 days, so a manager works from one list instead of running two separate manual checks.

5.1.7 Corporate Branch Overview

CORPORATE BRANCH OVERVIEW			
Regional Corporate Auditor / Franchise Owner · All Branches			
REGIONAL ROLLUP ● LIVE · Fri, 10 Jul 2026			
branch_product_summary_view — catalog breadth & stock by branch			
STORE ID	BRANCH	PRODUCTS STOCKED	TOTAL STOCK QTY
1	7-Eleven Siam Paragon	15	568
2	7-Eleven Silom Soi 2	14	1,142
3	7-Eleven Sukhumvit 11	11	944
4	7-Eleven Asoke Junction	11	835
5	7-Eleven Thonglor Soi 10	11	679
branch_staff_summary_view — headcount by branch and role			
STORE ID	BRANCH	ROLE	STAFF COUNT
1	7-Eleven Siam Paragon	CASHIER	3
1	7-Eleven Siam Paragon	MANAGER	1
1	7-Eleven Siam Paragon	RESTOCKER	1
1	7-Eleven Siam Paragon	SECURITY	1
2	7-Eleven Silom Soi 2	CASHIER	3

Views used: branch_product_summary_view, branch_staff_summary_view

A regional auditor or franchise owner needs the view none of the earlier screens give: every other screen in this report is scoped to a single store, so this one rolls catalog breadth, total stock, and staffing levels up across every branch into a single comparison table.

5.1.8 Sales History Lookup

<

View used: cashier_sales_view

The point-of-sale screen is for ringing up a new sale; this is for looking one up afterward. A cashier or store manager can resolve a customer question or process a return by transaction ID, product, or date, without touching the database directly.

5.1.9 Shift Summary Report

<

View used: employee_shift_summary_view

This is the payroll-review counterpart to the Employee Schedule & Hours screen: completed shifts with hours already computed, aggregated by branch rather than by individual employee, so a manager can reconcile a whole branch's timesheet in one pass instead of going employee by employee.

6. Security, Roles & Data Integrity

Security was implemented at three layers: constraint-based validation on every table, role-based access control restricting each staff tier to only the data it needs, and an audit trail that captures who changed what and when.

6.1 Role-Based Access Control



ROLES

```
1 CREATE USER bkk_register_01 IDENTIFIED BY 'pass2026';
2 CREATE ROLE pos_cashier_role;
3 GRANT ROLE pos_cashier_role TO bkk_register_01;
4 CREATE ROLE corporate_admin_role;
5 GRANT SELECT ON inventory_stock_view, cashier_sales_view TO pos_cashier_role;
6 GRANT INSERT ON sales_transactions, transaction_items TO pos_cashier_role;
7 GRANT UPDATE (points_balance) ON all_members TO pos_cashier_role;
8 GRANT pos_cashier_role TO store_manager_role;
9
10 GRANT SELECT ON
11   incoming_deliveries_view,
12   manager_inventory_alerts,
13   manager_daily_financials,
14   manager_daily_expenses,
15   manager_daily_net_profit,
16   branch_product_summary_view,
17   member_purchase_summary_view,
18   employee_shifts_summary_view,
19   active_staff_view,
20   branch_staff_summary_view
21 TO store_manager_role;
22
23 GRANT INSERT, UPDATE ON store_inventory, orders, deliveries, delivery_items TO store_manager_role;
24 GRANT INSERT, UPDATE ON employees, worker_shifts TO store_manager_role;
25 GRANT INSERT ON store_expenses TO store_manager_role;
26
27 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO corporate_admin_role;
28
29 CREATE USER bkk_register_01 WITH LOGIN PASSWORD 'pos terminal secure pass 2026';
30 GRANT ROLE pos_cashier_role TO bkk_register_01;
31 GRANT pos_cashier_role TO bkk_register_01;
```

Role	Access Granted
pos_cashier_role	SELECT on inventory_stock_view, cashier_sales_view; INSERT on sales_transactions, transaction_items; UPDATE (points_balance only) on all_members
store_manager_role	Inherits pos_cashier_role; SELECT on all manager/reporting views; INSERT/UPDATE on store_inventory, orders, deliveries, delivery_items, employees, worker_shifts; INSERT on store_expenses
corporate_admin_role	SELECT, INSERT, UPDATE, DELETE on all tables in the public schema

This was verified against two live login users. bkk_register_01 (granted pos_cashier_role) is correctly blocked from reading branches, all_members, categories, and manager-only views, while succeeding on the cashier's own sales screen:



ROLES - bkk_register_01

Authentication

Authentication:

Username:

Password: ☒ Save password

```
1 GRANT SELECT ON inventory_stock_view, cashier_sales_view TO pos_cashier_role;
2 GRANT INSERT ON sales_transactions, transaction_items TO pos_cashier_role;
3 GRANT UPDATE (points_balance) ON all_members TO pos_cashier_role;
4
5 CREATE USER bkk_register_01 WITH LOGIN PASSWORD 'pos terminal secure pass 2026';
6 GRANT pos_cashier_role TO bkk_register_01;
```

SQL Error [42501]: ERROR: permission denied for table branches

SQL Error [42501]: ERROR: permission denied for table all_members

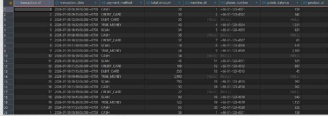
SQL Error [42501]: ERROR: permission denied for table categories

SQL Error [42501]: ERROR: permission denied for table deleted_products_audit

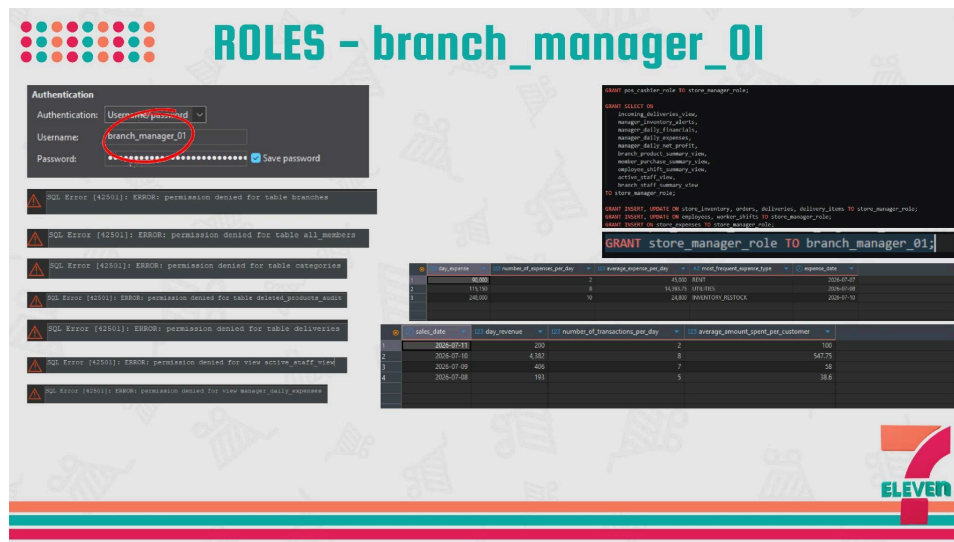
SQL Error [42501]: ERROR: permission denied for table deliveries

SQL Error [42501]: ERROR: permission denied for view active_staff_view

SQL Error [42501]: ERROR: permission denied for view manager_daily_expenses



branch_manager_01 (granted store_manager_role) is similarly blocked from all_members and other cashier/corporate-scoped tables, while succeeding on manager-level financial and operational views:



6.2 Data Integrity via Constraints

Every table enforces primary/foreign keys plus CHECK constraints appropriate to its data. Representative examples:

Table	Constraint	Business Rule Enforced
products	chk_positive_price	Price must be greater than 0
store_inventory	chk_positive_stock_quantity, chk_future_expiration_date	Stock cannot go negative; batches cannot be entered already expired
sales_transactions	chk_valid_payment_method	Payment method restricted to CASH, CREDIT_CARD, DEBIT_CARD, TRUE_MONEY, SCAN
all_members	chk_positive_points_balance, phone format regex	Loyalty points cannot go negative; phone numbers must match a valid format
employees	chk_valid_role_type	Role restricted to CASHIER, RESTOCKER, JANITOR, SECURITY, MANAGER
store_expenses	chk_valid_expense_type, chk_positive_amount	Expense category restricted to a fixed enum; amounts cannot be negative
worker_shifts	chk_valid_shift_status, chk_valid_shift_timeline	Status restricted to a fixed enum; clock_out cannot precede clock_in

We chose to enforce these at the database level rather than rely on application-side validation alone. Multiple staff roles and the audit/reporting layer all write to the same tables, so putting the rule in one place, the database, means no client can accidentally skip the check.

6.3 Audit Trail

Four dedicated audit tables (price_history_audit, deleted_products_audit, expenses_audit, transaction_audit) are populated by AFTER triggers on their corresponding operational tables. Each entry records the old value, the new value, the database user who made the change (current_user), and a timestamp. A regional auditor gets a

tamper-evident history that exists independently of any application-layer logging and can't be skipped by a client, which is what the project's security-mindset requirement is really asking for.

7. Performance & Optimization

Several design choices specifically target query efficiency for the highest-frequency access patterns: checkout, low-stock alerts, and manager dashboards.

- Composite primary keys double as indexes: `store_inventory`, `delivery_items`, and `transaction_items` use `(id, product_id, expiration_date)` as their primary key, which Postgres backs with a unique B-tree index automatically. This is exactly the lookup pattern used at checkout ("is this product in stock at this store, in which batch"), so no separate index is needed for that hot path.
- Pre-aggregated reporting views: `manager_daily_financials`, `manager_daily_expenses`, and `manager_daily_net_profit` group and sum at the daily grain once, rather than requiring the manager dashboard to re-run multi-way joins and aggregates across `sales_transactions` and `store_expenses` on every page load, across 20 branches.
- Trigger-maintained stock counters: `store_inventory.stock_quantity` is kept as a running total through triggers rather than computed on read as `SUM(quantity)` from transaction history. That costs one extra UPDATE per sale but saves considerably at read time, and it's the right trade given how the numbers work out: stock level gets read on every product page view and every low-stock check, versus written once per sale.
- One recommendation for later: PostgreSQL doesn't automatically index foreign key columns unless they're already part of a primary key. `sales_transactions.store_id`, `sales_transactions.employee_id`, and `employees.store_id` all fall into that gap, and manager dashboards filter and join on them often. Adding explicit B-tree indexes on these before the dataset grows much past its current seed size would be a sensible next step.

8. Deployment & Repository

8.1 Live Database (render.com)

External Link	postgresql://db_7_11_database_project_user:fWYIVuWgF5YB7EVjfk5ToKxhLTiNzEXv@dpg-d93o20mq1p3s73a08gog-a.oregon-postgres.render.com/db_7_11_database_project
Host Name	dpg-d93o20mq1p3s73a08gog-a
Port	5432
Database	db_7_11_database_project
Username	db_7_11_database_project_user
Password	fWYIVuWgF5YB7EVjfk5ToKxhLTiNzEXv

8.2 GitHub Repository

All schema, seed, function, trigger, view, role, and audit scripts are maintained at:

<https://github.com/yushtr/7-Eleven-Database-Foundations-Project>